ALGORITHM
Spring 2023, Final Exam, June 1
5:00-8:00 pm

1. (20%) Please answer the following questions:

   (a) Illustrate the operation of RADIX-SORT on the following numbers: 457, 654, 578, 254, 957, 499, 558, 911, 731, 536.

   (b) Show that if a node in a binary search tree has two children, then its successor has no left child.

   (c) Use induction to prove that radix sort works. Where does your proof need the assumption that the intermediate sort is stable?

   (d) Consider inserting the keys $7, 27, 19, 15, 27, 23, 11, 26$ into a hash table of length $m = 13$ using open addressing with the auxiliary hash function $h'(k) = k \mod 13$. Show the results after the insertion of each number using the linear hashing.

2. (20%) A red-black tree is a balanced binary tree that has height $O(\log n)$.

   (a) Show the red-black trees that result after successively inserting the keys $5, 22, 13, 15, 27, 33, 11, 21$ into an initially empty red-black tree.

   (b) What is the largest possible number of internal nodes in a red-black tree with black height k? What is the smallest possible number?

3. (20%) Medians and Order Statistics.

   (a) Design an algorithm to find both the minimum and the maximum of a set of n elements with $3\lfloor n/2 \rfloor$ comparisons in the worse case.

   (b) In the algorithm SELECT, the input elements are divided into groups of 5. Will the algorithm work in linear time if they are divided into groups of 7?

4. (20%) Minimizing lateness problem.

   - Single resource processes one job at a time.
   - Job i requires t_i units of processing time and is due at time d_i.
   - If i starts at time s_i, it finishes at time $f_i = s_i + t_i$.
   - Lateness: $\ell_i = \max\{0, f_i - d_i\}$.
   - Goal: schedule all jobs to minimize maximum lateness $L = \max\{\ell_i \mid 1 \le i \le n\}$.

i	1	2	3	4	5	6	7
t_i	3	6	4	1	2	7	5
d_i	10	12	9	8	14	13	11

5. (20%) Determine the cost and structure of an optimal binary search tree for a set of $n = 5$ keys with the following probabilities:

i	0	1	2	3	4	5
p_i		0.15	0.05	0.10	0.10	0.20
q_i	0.10	0.05	0.05	0.05	0.05	0.10

6. (20%) Consider the longest increasing subsequence problem defined as follows. Given a list of numbers $a_1, \ldots, a_n$ an increasing subsequence is a list of indices $i_1, \ldots, i_k \in \{1, \ldots, n\}$ such that $i_1 < i_2 < \ldots < i_k$ and $a_{i_1} \leq a_{i_2} \leq \ldots \leq a_{i_k}$. The longest increasing subsequence is the longest list of indices with this property.

   (a) What is the longest increasing subsequence of the list $5, 3, 4, 8, 7, 10$?

   (b) Consider the greedy algorithm that chooses the first element of the list, and then repeatedly chooses the next element that is larger. Is this a correct algorithm? Either prove its correctness or provide a counter example.

   (c) Consider a divide and conquer strategy that splits the list into the first half and second half, recursively computes $L = (\ell_1, \ldots, \ell_{k_L})$, $R = (r_1, \ldots, r_{k_r})$ the longest increasing subsequences in each half, and then, if the last chosen element in the first half is less than the first chosen index in the second half (i.e. $a_{\ell_{k_L}} \leq a_{r_1}$) returns $L \cdot R$, otherwise it returns the longer of L and R. Is this a correct algorithm? either prove its correctness of provide a counterexample.

   (d) Design a dynamic programming algorithm for longest increasing subsequence. Prove its correctness and analyze its running time.